

DATAVALIDERING MED



OVALIDERAD DATA LEDER TILL DÅLIGA ANALYSER OCH BESLUT

Datavalidering säkerställer att data har rätt struktur, typ och värden innan den används i analyser eller modeller.

Fel som upptäcks tidigt i dataflödet sprider sig inte vidare till nästa steg.

Jag valde Pandera eftersom det är byggt för dataframe validering och stödjer både kolumnkontroller och mer avancerade regler mellan kolumner.

Schema

Ett kontrakt som beskriver hur data förväntas se ut (t.ex. vilka kolumner som finns, vilken datatyp de ska ha, vilka värden är tillåtna).

Check

En regel som tillämpas på en eller flera kolumner (t.ex. värdena i en kolumn måste vara positiva).

Lazy validation

En inställning som tillåter att validering körs från början till slutet, medans alla fel samlas i en Error Report.

Pipeline-integration

Ett smidigt sätt för att integrera validering direkt i pipelinefunktioner genom dekoratorer (t.ex. `check_input`, `check_output`), så att data kontrolleras automatiskt.

**ETT SCHEMA
SAMLAR CHECKS,
VALIDERAS LAZILY,
OCH KAN KÖRAS
SOM ETT STEG I PIPELINEN**

ETT SCHEMA SOM VALIDERAR RÅ E-POSTKAMPANJDATA I ETT PROOF OF CONCEPT

Jag byggde:

- **Ett schema som validerar rå e-postkampanjdata:** rätt datatyper, giltiga ID:n, korrekta e-postadresser, tillåtna värden, samt rimliga datumrelationer mellan utskick, öppning, klick och transaktion.
- **Ett proof of concept där rå data läses in > valideras > felaktiga rader loggas och tas bort > datan valideras på nytt**

Jag skapade **syntetisk data** eftersom riktig kampanjdata inte är tillgänglig av integritetsskäl: **100 rader, 30% medvetet innehåller fel** (t.ex. negativa ID:n, felaktig e-post eller ett klick som sker före ett öppnat mail).

De felen loggades separat i en fil, **validation_errors.csv**, för att kunna jämföra dem med valideringsresultatet.

AV 100 RADER, UPPTÄCKTES 33 SOM FELAKTIGA

Programmet körs och skriver ut meddelanden i terminalen, så att varje steg i flödet kan följas.

Den första valideringen upptäcker 33 felaktiga rader och loggar dem i output/**rejected_rows_log.csv**.

Programmet tar därefter bort dessa rader och validerar den rensade datan på nytt, vilket bekräftar att allt nu är korrekt, och sparar resultatet i output/validated_rows.csv.

Genom att jämföra **validation_errors.csv** med **rejected_rows_log.csv** går det att bekräfta att **alla fel fångades**.

```
Validation_with_Pa...  main.py M X
> __pycache__
> .venv
> data
  email_marketing_dataset.csv
  validation_errors.csv
  output
    rejected_rows_log.csv M
    validated_rows.csv M
  .gitignore
  main.py M
  pandera_exploration.ipynb
  README.md
  report.md
  requirements.txt
  validation_schema.py

55 def main():
56
57     raw_df = pd.read
58     print("Starting
59

Problems Output Debug Console Ter

Terminal
(.venv) irene@Pascal:~/Desktop/09 Fördjupning i Pythonprogrammering/Validation_with_Pandera$
python main.py
Starting validation of raw data.
33 of 100 rows failed validation.
Rejection log written to output/rejected_rows_log.csv
67 rows passed validation after dropping rejected.
Validated data written to output/validated_rows.csv
(.venv) irene@Pascal:~/Desktop/09 Fördjupning i Pythonprogrammering/Validation_with_Pandera$
```

COLUMN CHECKS

```
"transaction_amount": pa.Column(
    float,
    pa.Check(is_valid_amount, element_wise=True),
    nullable=True,
),
```

Column checks **definieras inuti Column-objektet**, och **opererar på en enskild kolumns serie av värden**.

Vid lazy validering, rapporteras **fel per index inom den specifika kolumnen**, vilket gör det enkelt att se exakt vilket värde i vilken rad som bröt mot regeln.

vs

DATAFRAME CHECKS

```
checks=[
    pa.Check(check_opened_after_send, error="opened_at before send_date"),
```

Dataframe checks **definieras på DataFrameSchema-nivå**, genom att skicka in en funktion via checks-argumentet, och tillåter mer **komplex logik som involverar relationer mellan flera kolumner**, t.ex. att ett opened_date inte ligger före send_date.

Vid lazy validering, rapporteras **fel utifrån dataframens radindex** men column-värdet pekar inte på en specifik ansvarig kolumn, så det är svårare att snabbt tolka dem.

VALIDERING I PIPELINEN FUNGERAR MEN SKALBARHET, STÄDNING OCH VERKLIG DATA ÅTERSTÅR

Det viktigaste jag lärde mig var hur man definierar ett schema och hur validering kan integreras i en pipeline för att automatisera kontrollen av data.

Tre begränsningar:

- Fokus på DataFrameSchema snarare än DataFrameModel, som är mer skalbart för större projekt.
- Parser-funktionalitet implementerades inte i själva proof of concept.
- Hela lösningen bygger på syntetisk data och en verklig dataset hade innehållit mer oväntade och komplexa felmönster.